

# Data Access and Storage with Python

Accessing, Importing, Integrating, and Storing Data for Analysis

DSAI1005 - Data Analysis with Python

Faculty of Data Science and Artificial Intelligence  
National Economics University

2026

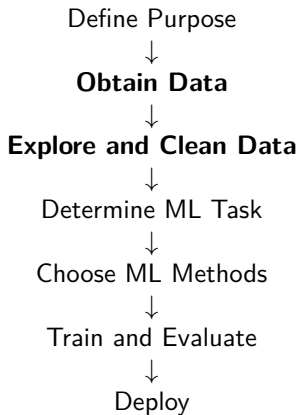
## Lesson Motivation

In Data Science and Machine Learning projects, data is rarely available as a clean and analysis-ready DataFrame.

Data may be stored in many formats:

- CSV and delimited text files
- Excel workbooks
- JSON returned by Web APIs
- HTML tables on websites
- PDF reports and documents
- relational databases and NoSQL databases

# Where This Topic Fits in the ML Workflow



This lesson focuses on the work that must happen before modeling begins:

- obtaining data;
- preparing data;
- integrating sources;
- storing reusable results.

# Learning Objectives

After this lesson, learners should be able to:

- access data from CSV, Excel, JSON, HTML, PDF, SQLite, and MongoDB;
- use key Pandas I/O parameters such as `usecols`, `dtype`, `parse_dates`, and `chunksize`;
- flatten nested JSON data into tabular form;
- move data between SQL databases and Pandas DataFrames;
- build a multi-source data pipeline;
- choose suitable storage formats for analytical tasks.

# Lesson Structure

- 1 Overview of data access and storage
- 2 CSV files with NumPy
- 3 CSV files with Pandas
- 4 Microsoft Excel
- 5 JSON
- 6 HTML and PDF
- 7 SQLite
- 8 MongoDB and PyMongo
- 9 Multi-source data pipelines
- 10 Best practices

# Prerequisites

To follow the lesson effectively, learners should have:

- basic Python knowledge;
- familiarity with variables, lists, dictionaries, loops, and functions;
- basic knowledge of NumPy and Pandas DataFrames;
- understanding of rows, columns, and tabular data;
- access to Jupyter Notebook, Google Colab, VS Code, or a similar environment.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# What Is Data Access?

**Data access** is the process of retrieving data from a storage source and bringing it into an analytical environment.

Example: loading a CSV file into Pandas.

```
customers.csv  
  |  
  v  
pd.read_csv()  
  |  
  v  
DataFrame
```

Example: querying a database before analysis.

```
SQL Database -> SQL Query -> Pandas DataFrame
```



# What Is Data Storage?

**Data storage** is the process of saving data or analytical results in a reusable format or storage system.

```
DataFrame -> to_csv() -> report.csv
```

```
DataFrame -> to_sql() -> Database Table
```

Storage is not only an output step. It also supports reproducibility, reporting, sharing, and later modeling.

# Common Data Sources

| Source Type          | Examples                  |
|----------------------|---------------------------|
| Flat files           | CSV, TSV                  |
| Spreadsheets         | Excel                     |
| Semi-structured data | JSON                      |
| Web data             | HTML tables               |
| Documents            | PDF reports               |
| Relational databases | SQLite, PostgreSQL, MySQL |
| NoSQL databases      | MongoDB                   |

# Structured and Semi-Structured Data

Structured data has a clear row-and-column structure.

| CustomerID | Name | City | Revenue |
|------------|------|------|---------|
|------------|------|------|---------|

Semi-structured data may contain nested objects and lists.

```
{
  "customer": {"name": "An", "city": "Hanoi"},
  "orders": [
    {"product": "A"},
    {"product": "B"}
  ]
}
```

## Exercise: Classify Data Sources

Classify each source into a suitable category.

- 1 sales.csv
- 2 customers.xlsx
- 3 JSON data returned by a Web API
- 4 company.db
- 5 MongoDB
- 6 A financial report in PDF format

Use: flat file, spreadsheet, semi-structured data, relational database, NoSQL database, document.

# Knowledge Check

**Question 1.** Which format is commonly considered a flat file?

- ☐ A. CSV
- ☐ B. MongoDB
- ☐ C. SQLite
- ☐ D. PDF

**Question 2.** Which format naturally supports nested objects and lists?

- ☐ A. JSON
- ☐ B. CSV
- ☐ C. TSV
- ☐ D. Numerical matrix

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

## Reading Numerical Files with `np.loadtxt()`

`np.loadtxt()` is suitable when data is mainly numerical, simple, and has no missing values.

```
import numpy as np

data = np.loadtxt(
    "matrix_data.csv",
    delimiter=",",
    skiprows=1
)

print(data)
print(data.shape)
```

Column means can be computed using:

```
print(np.mean(data, axis=0))
```

## Exercise: `np.loadtxt()`

Complete the program to read `sales.csv`, where values are separated by commas.

```
# data = np.loadtxt(  
# "sales.csv",  
# delimiter=...  
# )  
  
# print(data.shape)
```

Then determine the number of rows and columns.



## Handling Missing Values with `np.genfromtxt()`

`np.genfromtxt()` is more flexible than `np.loadtxt()` when numerical files contain missing values.

```
data = np.genfromtxt(  
    "numeric_data.csv",  
    delimiter=",",  
    skip_header=1,  
    filling_values=0.0,  
    dtype=float  
)  
  
print(data)
```

Here, missing values are replaced with 0.0.

## Writing Arrays with `np.savetxt()`

`np.savetxt()` saves a NumPy array to a text file.

```
output = np.random.randn(5, 3)

np.savetxt(
    "output.csv",
    output,
    delimiter=",",
    fmt="%.2f",
    header="A,B,C",
    comments=""
)
```

`fmt="%.2f"` writes each number with two decimal places.

# NumPy CSV Functions

| Function                     | Purpose                                               |
|------------------------------|-------------------------------------------------------|
| <code>np.loadtxt()</code>    | Read numerical data from simple structured text files |
| <code>np.genfromtxt()</code> | Read numerical data with missing-value support        |
| <code>np.savetxt()</code>    | Write NumPy arrays to text files                      |

## Practical Exercise: Monthly Sales

Create `monthly_sales.csv` with the following content:

```
Month,Revenue,Cost
1,120,80
2,150,
3,180,110
4,,130
```

Complete these tasks:

- 1 Read the data using `np.genfromtxt()`.
- 2 Replace missing values with 0.
- 3 Calculate mean Revenue and Cost.
- 4 Save the processed numerical data using `np.savetxt()`.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas**
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

## Reading CSV Files with `pd.read_csv()`

Pandas provides `pd.read_csv()` for reading tabular data from CSV files.

```
import pandas as pd

df = pd.read_csv("customers.csv")

print(df.head())
```

Pandas is suitable when data contains text, dates, multiple types, missing values, and column labels.

## Specifying Delimiters with `sep`

Not every file uses a comma as the separator.  
Semicolon-delimited file:

```
df = pd.read_csv(  
    "data.csv",  
    sep=";"  
)
```

Tab-separated file:

```
df = pd.read_csv(  
    "data.tsv",  
    sep="\t"  
)
```

## Reading Selected Columns with `usecols`

`usecols` selects columns during import.

```
df = pd.read_csv(  
    "customers.csv",  
    usecols=[  
        "customer_id",  
        "city",  
        "total_spent"  
    ]  
)
```

Benefits include lower memory use, faster loading, and simpler downstream analysis.



## Specifying Data Types with dtype

Pandas can infer data types automatically, but explicit types are often safer.

```
df = pd.read_csv(  
    "customers.csv",  
    dtype={  
        "customer_id": str,  
        "age": "Int64"  
    }  
)
```

Fields such as CustomerID, StudentID, ProductID, and PostalCode usually represent identifiers, not numerical measurements.

## Parsing Dates with `parse_dates`

Pandas can convert columns to datetime during import.

```
df = pd.read_csv(  
    "orders.csv",  
    parse_dates=["OrderDate"]  
)  
  
print(df.dtypes)
```

This is useful before filtering, grouping, or resampling by time.

## Defining Missing Values with `na_values`

Real datasets may use different symbols for missing values.

```
df = pd.read_csv(  
    "customers.csv",  
    na_values=[  
        "NA",  
        "N/A",  
        "-",  
        "missing"  
    ]  
)
```

Pandas converts these symbols into standard missing values.

# Character Encoding

Use UTF-8 when reading text with non-ASCII characters.

```
df = pd.read_csv(  
    "customers.csv",  
    encoding="utf-8"  
)
```

For CSV files intended for Microsoft Excel on Windows:

```
df.to_csv(  
    "customers_clean.csv",  
    index=False,  
    encoding="utf-8-sig"  
)
```

# Inspecting Data After Import

After loading a dataset, inspect structure and quality immediately.

```
df.head()  
df.shape  
df.info()  
df.dtypes  
df.isna().sum()
```

These commands help identify row counts, column names, data types, non-missing values, and missing-value counts.

# Reading Large Datasets with chunksize

When a CSV file is larger than available memory, process it incrementally.

```
total = 0

for chunk in pd.read_csv(
    "transactions.csv",
    chunksize=100000
):
    total += chunk["Amount"].sum()

print(total)
```

The file is read, processed, and discarded one chunk at a time.

# Writing a DataFrame to CSV

Use `to_csv()` to export a DataFrame.

```
df.to_csv(  
    "clean_data.csv",  
    index=False,  
    encoding="utf-8-sig"  
)
```

`index=False` prevents the DataFrame index from being written as an additional column.

# Pandas CSV Functions and Arguments

| Function or Argument       | Purpose                           |
|----------------------------|-----------------------------------|
| <code>pd.read_csv()</code> | Read CSV or other delimited files |
| <code>usecols=</code>      | Read selected columns only        |
| <code>dtype=</code>        | Specify data types                |
| <code>parse_dates=</code>  | Convert columns to datetime       |
| <code>na_values=</code>    | Define missing-value symbols      |
| <code>chunksize=</code>    | Read data incrementally           |
| <code>df.to_csv()</code>   | Write a DataFrame to CSV          |



## Knowledge Check: Pandas CSV

**Question 1.** Which argument allows only selected columns to be read?

- ☐ A. usecols
- ☐ B. keepcols
- ☐ C. columns\_only
- ☐ D. filter\_columns

**Question 2.** Which argument is useful for very large CSV files?

- ☐ A. chunksize
- ☐ B. batch=True
- ☐ C. split=True
- ☐ D. large=True

## Practical Exercise: Customer CSV

Create `customers.csv` containing `CustomerID`, `Name`, `City`, `Age`, `SignupDate`, and `TotalSpent`.

Include missing symbols such as `NA` and `-`, and `CustomerID` values such as `001` and `002`.

Tasks:

- 1 Read `CustomerID` as `str`.
- 2 Parse `SignupDate` as `datetime`.
- 3 Treat `NA` and `-` as missing values.
- 4 Read only five columns needed for analysis.
- 5 Inspect the data and export `customers_clean.csv`.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel**
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# Reading a Worksheet

Pandas uses `pd.read_excel()` to read Excel data.

```
df = pd.read_excel(  
    "products.xlsx",  
    sheet_name="Products",  
    engine="openpyxl"  
)
```

The `sheet_name` argument specifies the worksheet to read.

# Reading All Worksheets

An entire workbook can be read by setting `sheet_name=None`.

```
workbook = pd.read_excel(  
    "products.xlsx",  
    sheet_name=None  
)  
  
print(workbook.keys())  
products = workbook["Products"]
```

The result is a dictionary of DataFrames.

# Writing Multiple DataFrames to One Workbook

Use `pd.ExcelWriter()` to write multiple DataFrames to separate worksheets.

```
with pd.ExcelWriter(
    "report.xlsx",
    engine="openpyxl"
) as writer:
    df_products.to_excel(
        writer,
        sheet_name="Products",
        index=False
    )
    df_sales.to_excel(
        writer,
        sheet_name="Sales",
        index=False
    )
```

# Excel Functions

| Function or Argument          | Purpose                                   |
|-------------------------------|-------------------------------------------|
| <code>pd.read_excel()</code>  | Read Excel data                           |
| <code>sheet_name=</code>      | Select a worksheet                        |
| <code>sheet_name=None</code>  | Read all worksheets                       |
| <code>df.to_excel()</code>    | Write a DataFrame to Excel                |
| <code>pd.ExcelWriter()</code> | Write multiple DataFrames to one workbook |

## Excel Exercise and Knowledge Check

**Exercise.** Create `business_data.xlsx` with three worksheets: Customers, Products, and Sales. Then read the workbook, inspect worksheet names and dimensions, calculate total Sales, and export a summary workbook.

**Question 1.** What does `pd.read_excel()` return when `sheet_name=None` is used?

- ☐ A. A dictionary of DataFrames
- ☐ B. Only the first worksheet
- ☐ C. A NumPy array
- ☐ D. A string



# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON**
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# What Is JSON?

JSON, or JavaScript Object Notation, is widely used in Web APIs, e-commerce systems, microservices, and NoSQL databases.

```
[  
  {"customer_id": "C001", "city": "Hanoi"},  
  {"customer_id": "C002", "city": "Danang"}  
]
```

For relatively flat JSON structures:

```
df = pd.read_json("customers.json")
```

# Nested JSON

Real-world JSON often contains multiple levels.

```
{
  "order_id": "0001",
  "customer": {
    "customer_id": "C001",
    "city": "Hanoi"
  },
  "items": [
    {"sku": "P001", "qty": 2},
    {"sku": "P002", "qty": 1}
  ]
}
```

For analysis, each item may need to become a separate row.

## Flattening Data with `pd.json_normalize()`

```
df_items = pd.json_normalize(  
    orders,  
    record_path=["items"],  
    meta=[  
        "order_id",  
        ["customer", "customer_id"],  
        ["customer", "city"]  
    ]  
)
```

- `record_path` specifies the nested list to expand into rows.
- `meta` specifies parent-level attributes to retain.

## JSON Exercise and Knowledge Check

**Exercise.** Create three orders in JSON. Each order contains OrderID, OrderDate, Customer, and Items. Flatten the data so that each item becomes one row, then calculate Revenue by City.

**Question.** What is the purpose of `record_path` in `pd.json_normalize()`?

- ☐ A. To identify the nested list to expand into rows
- ☐ B. To rename columns
- ☐ C. To write data to JSON
- ☐ D. To remove missing values

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF**
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# Reading HTML Tables

`pd.read_html()` detects HTML `<table>` elements and converts them into DataFrames.

```
tables = pd.read_html(  
    "financial_quotes.html"  
)  
  
print(len(tables))  
stocks = tables[0]
```

The result is a list of DataFrames.

# Why PDF Extraction Is Harder

CSV is designed to represent tabular data.

PDF is primarily designed to:

- display content;
- preserve layout;
- support printing.

Rows and columns that appear visually may not be stored internally as explicit table structures.



# Extracting Tables with pdfplumber

```
import pdfplumber

with pdfplumber.open("report.pdf") as pdf:
    page = pdf.pages[0]
    tables = page.extract_tables()
```

Convert one extracted table to a DataFrame:

```
table = tables[0]

df = pd.DataFrame(
    table[1:],
    columns=table[0]
)
```

# HTML and PDF Exercise

Given an HTML file containing a product-price table and a foreign-exchange table:

- ① read both tables into Pandas;
- ② inspect the structure of each table;
- ③ save each table to a separate worksheet in one Excel workbook.

For a table extracted from a PDF, inspect headers, missing values, and required cleaning operations before analysis.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases**
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# What Is SQLite?

SQLite is a lightweight relational database system. An entire database can be stored in one file, such as `company.db`.

Python provides SQLite support through the standard library.

```
import sqlite3
```

Create a connection and cursor:

```
conn = sqlite3.connect("company.db")  
cursor = conn.cursor()
```

## Executing SQL Queries

The following query retrieves employees with salary at least 30 million.

```
cursor.execute(  
    """  
    SELECT emp_id, full_name, salary  
    FROM employees  
    WHERE salary >= ?  
    """,  
    (30000000,) )  
  
rows = cursor.fetchall()
```

`fetchall()` returns all rows satisfying the query.

# Parameterized Queries

When query values come from variables or user input, use parameters.

```
cursor.execute(  
    "SELECT * FROM users WHERE id = ?",  
    (user_id,) )
```

This separates SQL structure from values and helps reduce security risks associated with direct string concatenation.

# Reading SQL Results into Pandas

Pandas can read SQL query results directly into a DataFrame.

```
query = """
SELECT
    emp_id,
    full_name,
    salary
FROM employees
"""

df = pd.read_sql_query(
    query,
    conn
)
```

# Combining Tables with JOIN

```
SELECT
    e.emp_id,
    e.full_name,
    d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

The query result can then be loaded into Pandas.

```
df = pd.read_sql_query(
    query,
    conn
)
```



# Writing DataFrames to SQLite

```
df.to_sql(  
    name="sales_summary",  
    con=conn,  
    if_exists="replace",  
    index=False  
)
```

Common values of `if_exists`:

- `fail`: raise an error if the table exists;
- `replace`: replace the existing table;
- `append`: add rows to the existing table.

# SQLite Functions and Exercise

| Function                         | Purpose                          |
|----------------------------------|----------------------------------|
| <code>sqlite3.connect()</code>   | Open a connection to SQLite      |
| <code>cursor.execute()</code>    | Execute an SQL statement         |
| <code>fetchall()</code>          | Retrieve query results           |
| <code>pd.read_sql_query()</code> | Convert SQL results to DataFrame |
| <code>df.to_sql()</code>         | Write DataFrame to SQL table     |

**Exercise.** Create `sales.db`, join `orders` and `customers`, calculate total Amount by City, and store the result in `city_sales_summary`.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo**
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices

# What Is MongoDB?

MongoDB is a document-oriented NoSQL database system.

Relational database:

Table  
-> Row

MongoDB:

Collection  
-> Document

Documents can contain scalar values, nested objects, lists, and multiple levels of hierarchy.

# Connecting to MongoDB

PyMongo provides MongoClient for connecting to MongoDB.

```
from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017/"
)

db = client["retail_db"]
orders = db["orders"]
```

# CRUD Operations

CRUD refers to Create, Read, Update, and Delete.

```
# Create
orders.insert_one(sample_order)

# Read one document
orders.find_one({"order_id": "ORD001"})

# Filter multiple documents
orders.find({"total_amount": {"$gte": 1000000}})

# Update
orders.update_one(
    {"order_id": "ORD001"},
    {"$set": {"status": "REFUNDED"}}
)
```

# Converting MongoDB Data to Pandas

Query results can be converted into a list of documents.

```
documents = list(
    orders.find()
)

df = pd.json_normalize(
    documents
)
```

MongoDB's `_id` field can be converted before exporting.

```
df["_id"] = df["_id"].astype(str)
```

# MongoDB Exercise and Knowledge Check

**Exercise.** Create a collection named `customer_orders` with at least five documents. Insert orders, query paid orders, update one status, convert results to Pandas, and calculate paid revenue.

**Question.** Which Pandas function is useful for nested MongoDB documents?

- ☐ A. `pd.json_normalize()`
- ☐ B. `pd.read_csv()`
- ☐ C. `pd.crosstab()`
- ☐ D. `pd.cut()`



# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline**
- 10 Best Practices

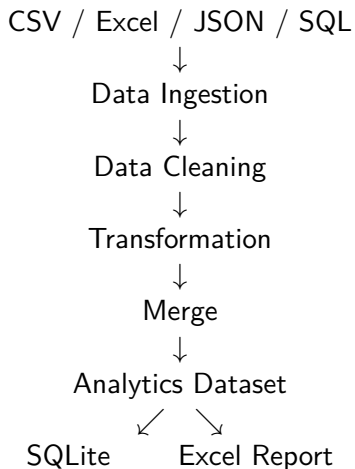
# Pipeline Scenario

A company stores data in multiple sources:

```
customers.csv  
products.xlsx  
orders.json  
company.db
```

The goal is to integrate these sources into one analytical dataset for reporting, visualization, or Machine Learning.

# General Integration Workflow



# Reading Multiple Sources

```
customers = pd.read_csv("customers.csv")

products = pd.read_excel(
    "products.xlsx",
    sheet_name="Products"
)

import json
with open("orders.json", "r", encoding="utf-8") as f:
    orders = json.load(f)
```

# Flattening Orders and Creating Variables

```
items = pd.json_normalize(  
    orders,  
    record_path=["items"],  
    meta=[  
        "order_id",  
        ["customer", "customer_id"]  
    ]  
)  
  
items["Revenue"] = (  
    items["Quantity"] * items["Price"]  
)
```

# Integrating Data with `merge()`

```
df = items.merge(  
    products,  
    on="ProductID",  
    how="left"  
)  
  
df = df.merge(  
    customers,  
    on="CustomerID",  
    how="left"  
)
```

After each merge, inspect:

```
print(df.shape)  
print(df.isna().sum())
```

# Creating KPI Summaries

Calculate revenue and number of orders by region.

```
region_summary = (  
    df  
    .groupby("Region")  
    .agg(  
        Revenue=("Revenue", "sum"),  
        Orders=("OrderID", "nunique")  
    )  
    .reset_index()  
)
```

# Storing and Reporting Results

Store analytical data in SQLite.

```
conn = sqlite3.connect("analytics.db")

df.to_sql(
    "order_analytics",
    conn,
    if_exists="replace",
    index=False
)

conn.close()
```

Export KPI summaries to Excel.

```
with pd.ExcelWriter("dashboard.xlsx") as writer:
    df.to_excel(writer, sheet_name="Details", index=False)
    region_summary.to_excel(writer, sheet_name="Region_KPI", index=False)
```



## Project Exercise: Multi-Source Integration

Build `order_analytics` from `customers.csv`, `products.xlsx`, and `orders.json`.

Required fields:

- `OrderID`, `CustomerID`, `CustomerName`, `City`
- `ProductID`, `Category`, `Quantity`, `UnitPrice`, `Revenue`

Tasks: read sources, inspect data, flatten JSON, merge tables, identify unmatched records, calculate revenue, save details to SQLite, and export KPI summaries to Excel.

# Roadmap

- 1 Overview of Data Access and Storage
- 2 Working with CSV Files Using NumPy
- 3 Working with CSV Files Using Pandas
- 4 Working with Microsoft Excel
- 5 Working with JSON
- 6 Accessing Data from HTML and PDF
- 7 Working with SQLite Databases
- 8 Working with MongoDB and PyMongo
- 9 Building a Multi-Source Data Pipeline
- 10 Best Practices**

## Read Only Required Columns

If a file contains many columns but only a subset is needed, select columns during import.

```
pd.read_csv(  
    "large.csv",  
    usecols=[  
        "CustomerID",  
        "Sales"  
    ]  
)
```

This reduces loading time and memory consumption.

# Inspect Early and Often

Inspect immediately after import.

```
df.head()  
df.shape  
df.info()  
df.isna().sum()
```

Inspect again after integration.

```
df.isna().sum()
```

New missing values may appear when keys do not match across sources.

# Control Data Types

Inspect data types using:

```
df.dtypes
```

Do not assume that every field containing digits is numerical. For example, `CustomerID = "001"` is an identifier and should often be stored as a string.

# Use Context Managers

When working with files, use `with` statements.

```
with open("data.json", encoding="utf-8") as f:  
    data = json.load(f)
```

For Excel reports:

```
with pd.ExcelWriter("report.xlsx") as writer:  
    summary.to_excel(writer, index=False)
```

Resources are closed properly after the operation is complete.

## Choose an Appropriate Storage Format

| Requirement                       | Suitable Format |
|-----------------------------------|-----------------|
| Exchange simple tabular data      | CSV             |
| Create multi-sheet reports        | Excel           |
| Exchange data through Web APIs    | JSON            |
| Store relational data and queries | SQL             |
| Store flexible nested documents   | MongoDB         |
| Store large tabular datasets      | Parquet         |
| Extract website tables            | HTML            |
| Distribute fixed-layout reports   | PDF             |

# Final Knowledge Check

**Question 1.** Which technique is appropriate when a CSV file is larger than RAM?

- ☐ A. Reading the file in chunks
- ☐ B. Converting all values to strings
- ☐ C. Creating a second copy of the file
- ☐ D. Using only `head()`

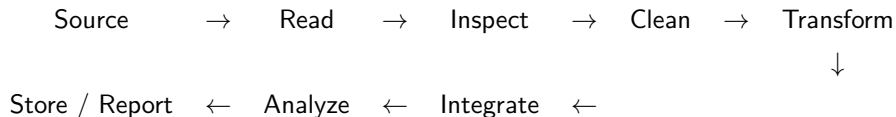
**Question 2.** Which format is especially common for exchanging data through Web APIs?

- ☐ A. JSON
- ☐ B. PDF
- ☐ C. XLS
- ☐ D. SQLite



# Summary

A general workflow for working with data from multiple sources:



The goal is to transform raw, distributed data into reliable analytical data.

# Key Tools

| Data Source      | Python Tools                                                                 |
|------------------|------------------------------------------------------------------------------|
| CSV              | <code>pd.read_csv()</code> , <code>to_csv()</code>                           |
| NumPy text files | <code>loadtxt()</code> , <code>genfromtxt()</code> , <code>savetxt()</code>  |
| Excel            | <code>read_excel()</code> , <code>ExcelWriter()</code>                       |
| JSON             | <code>read_json()</code> , <code>json_normalize()</code>                     |
| HTML             | <code>read_html()</code>                                                     |
| PDF              | <code>pdfplumber</code>                                                      |
| SQLite           | <code>sqlite3</code> , <code>read_sql_query()</code> , <code>to_sql()</code> |
| MongoDB          | <code>pymongo</code> , <code>json_normalize()</code>                         |

## Closing Message

Data access, preparation, and storage are not secondary technical steps.

They determine whether later analysis, visualization, and Machine Learning models can be trusted.

Correct data pipeline  $\Rightarrow$  reliable analytical results